

Design and Analysis of Algorithms + **Data Structures**

Comprehensive Study Notes for GATE Computer Science

Ankur Gupta

www.ankurgupta.net

Note: These notes have been prepared primarily from the following books:

- (1) **Introduction to Algorithms** by Cormen, Leiserson, Rivest, Stein
- (2) **Data Structures** by Seymour Lipschutz

Please refer to the above books for more details.

Contents

1	Growth of Functions	6
1.1	Asymptotic Notation	6
1.1.1	Big-O Notation (Asymptotic Tight Bound)	6
1.1.2	Big-O Notation (Asymptotic Upper Bound)	6
1.1.3	Big-Omega Notation (Asymptotic Lower Bound)	6
1.1.4	Small-o Notation (Upper Bound that is not Asymptotically Tight)	7
1.1.5	Small-omega Notation (Lower Bound that is not Asymptotically Tight)	7
1.1.6	Important Results	7
1.1.7	Comparison of Functions	7
1.2	Comparison Tables	8
1.3	Important Examples	8
1.3.1	Polynomial Functions	8
1.4	Logarithms	8
1.4.1	Properties of Logarithms	8
1.4.2	Iterated Logarithm Function	8
2	Recurrences	9
2.1	Master Method	9
2.1.1	Solution Cases	9
2.2	Changing Variables	9
3	Sorting Algorithms	10
3.1	Insertion Sort	10
3.1.1	Algorithm Description	10
3.2	Merge Sort	10
3.2.1	Algorithm Description	10
3.3	Bubble Sort	10
3.3.1	Algorithm Description	10
3.4	Selection Sort	11
3.4.1	Algorithm Description	11
3.5	Heap Sort	11
3.5.1	Max-Heapify	11
3.5.2	Build-Max-Heap	11
3.5.3	Heapsort	11
3.5.4	Heap-Extracts-Max	11
3.5.5	Heap-Increase-Key	12
3.5.6	Max-Heap-Insert	12

3.6	Quick Sort	12
3.6.1	Algorithm Description	12
3.6.2	Performance Analysis	12
3.7	Counting Sort	13
3.7.1	Algorithm Description	13
3.8	Radix Sort	13
3.8.1	Algorithm Description	13
3.9	Bucket Sort	13
3.9.1	Algorithm Description	13
4	Graph Algorithms	14
4.1	Representation of Graphs	14
4.1.1	Adjacency List Representation	14
4.1.2	Adjacency Matrix Representation	14
4.2	Breadth-First Search	14
4.2.1	Algorithm Description	15
4.3	Depth-First Search	15
4.3.1	Contains Two Methods	15
4.4	Topological Sorting	15
4.5	Strongly Connected Components	15
4.6	Minimum Spanning Trees	16
4.6.1	Kruskal's Algorithm	16
4.6.2	Prim's Algorithm	16
4.7	Single Source Shortest Paths	17
4.7.1	Bellman-Ford Algorithm	17
4.7.2	Dijkstra's Algorithm	17
5	Data Structures	18
5.1	Arrays	18
5.1.1	Representation of Linear Array in Memory	18
5.1.2	Representation of 2-D Array in Memory	18
5.2	Stacks	19
5.2.1	Arithmetic Expressions	19
5.3	Queues	19
5.4	Linked Lists	19
5.5	Binary Trees	20
5.5.1	Definition	20
5.5.2	Complete Binary Trees	20
5.5.3	Extended Binary Tree (2-Tree)	20
5.6	Binary Search Trees	20
5.6.1	Operations on a BST	20
5.7	AVL Search Trees	21
5.7.1	Representation	21
5.7.2	Insertion in AVL Search Tree	21
5.7.3	Types of Rotations	21
5.7.4	LR and RL Rotations	22
5.8	B-Trees	22
5.8.1	Minimum degree of the B-tree	22

5.8.2	Height of a B-tree	22
5.8.3	Searching a B-Tree	23
5.8.4	Inserting a Key into a B-Tree	23
5.8.5	Deleting a Key from a B-Tree	23
5.9	Hash Tables	24
5.9.1	Direct Address Tables	24
5.9.2	Hash Tables	24
5.9.3	Hash Functions	24
5.9.4	Types of Hashing Schemes	24
5.9.5	Collision Resolution	24
5.9.6	Types of Probing	25
5.9.7	Perfect Hashing	25
6	Additional Topics	26
6.1	Tower of Hanoi Problem	26
6.1.1	Algorithm	26
6.2	Assembly Line Scheduling	26
6.3	Longest Common Subsequence	26
6.4	Matrix Chain Multiplication	27
6.4.1	Solution	27
6.4.2	Inversions	27

Chapter 1

Growth of Functions

1.1 Asymptotic Notation

Asymptotic notation provides a framework for analyzing the efficiency of algorithms. We use these notations to describe the limiting behavior of a function when the argument tends toward infinity.

1.1.1 Big-O Notation (Asymptotic Tight Bound)

Definition 1.1.1 (Big-O Notation). A function $f(n)$ belongs to $\mathcal{O}(g(n))$ if there exist positive constants c_1 , c_2 , and n_0 such that:

$$0 \leq c_1 g(n) \leq f(n) \leq c_2 g(n) \text{ for all } n \geq n_0$$

This means that $f(n)$ is bounded both above and below by $g(n)$ for sufficiently large n .

Key properties:

- $f(n) = \Theta(g(n))$ if and only if $f(n) = \mathcal{O}(g(n))$ and $f(n) = \Omega(g(n))$
- $\mathcal{O}(g(n)) \cap \Omega(g(n)) = \Theta(g(n))$

1.1.2 Big-O Notation (Asymptotic Upper Bound)

Definition 1.1.2 (O-notation). A function $f(n)$ belongs to $\mathcal{O}(g(n))$ if there exist positive constants c and n_0 such that:

$$0 \leq f(n) \leq c \cdot g(n) \text{ for all } n \geq n_0$$

Example: $f(n) = O(n^2)$ means that the worst-case running time is $O(n^2)$.

1.1.3 Big-Omega Notation (Asymptotic Lower Bound)

Definition 1.1.3 (Ω -notation). A function $f(n)$ belongs to $\Omega(g(n))$ if there exist positive constants c and n_0 such that:

$$0 \leq c \cdot g(n) \leq f(n) \text{ for all } n \geq n_0$$

This means that for all values of n greater than or equal to n_0 , the value $f(n)$ is on or above $c \cdot g(n)$.

1.1.4 Small-o Notation (Upper Bound that is not Asymptotically Tight)

Definition 1.1.4 (o-notation). A function $f(n)$ belongs to $o(g(n))$ if for any positive constant $c > 0$, there exists a constant $n_0 > 0$ such that:

$$0 \leq f(n) < c \cdot g(n) \text{ for all } n \geq n_0$$

1.1.5 Small-omega Notation (Lower Bound that is not Asymptotically Tight)

Definition 1.1.5 (ω -notation). A function $f(n)$ belongs to $\omega(g(n))$ if for any positive constant $c > 0$, there exists a constant $n_0 > 0$ such that:

$$0 \leq c \cdot g(n) < f(n) \text{ for all } n \geq n_0$$

1.1.6 Important Results

#1 $f(n) = \Theta(g(n))$ if and only if $f(n) = \mathcal{O}(g(n))$ and $f(n) = \Omega(g(n))$

#2 $\mathcal{O}(g(n)) \cap \Omega(g(n)) = \Theta(g(n))$

#3 $\mathcal{O}(g(n)) \cap \omega(g(n)) = \emptyset$

1.1.7 Comparison of Functions

The following relationships hold for comparing asymptotic growth rates:

Transitivity:

◦ If $f(n) = \Theta(g(n))$ and $g(n) = \Theta(h(n))$, then $f(n) = \Theta(h(n))$

Reflexivity:

$$f(n) = \mathcal{O}(f(n))$$

$$f(n) = \Omega(f(n))$$

$$f(n) = \Theta(f(n))$$

Symmetry:

$$f(n) = \Theta(g(n)) \iff g(n) = \Theta(f(n))$$

Transpose Symmetry:

$$f(n) = \mathcal{O}(g(n)) \iff g(n) = \Omega(f(n))$$

$$f(n) = o(g(n)) \implies g(n) = \omega(f(n))$$

Condition	Relation
$f(n) = \mathcal{O}(g(n))$	$a \leq b$
$f(n) = \Omega(g(n))$	$a \geq b$
$f(n) = \Theta(g(n))$	$a = b$
$f(n) = o(g(n))$	$a < b$
$f(n) = \omega(g(n))$	$a > b$

Table 1.1: Analogy between asymptotic comparisons and numerical comparisons

1.2 Comparison Tables

1.3 Important Examples

1.3.1 Polynomial Functions

Example 1.3.1. Consider $(n + a)^b = \Theta(n^b)$ where a and b are constants and $b > 0$.

Example 1.3.2. $2^{n+1} = \mathcal{O}(2^n)$ but $2^{2n} \neq \mathcal{O}(2^n)$

1.4 Logarithms

1.4.1 Properties of Logarithms

- (1) $\log_b a^k = k \log_b a$
- (2) $\log_b \log_c n = \log_b (\log_c n)$
- (3) $\log_b (n^a) = a \log_b n$
- (4) $\log_b a = \frac{1}{\log_a b}$
- (5) $\frac{\log_a c}{\log_b c} = \frac{\log_a b}{1}$

1.4.2 Iterated Logarithm Function

The iterated logarithm function is defined as:

$$\lg^* n = \min\{i \geq 0 : \lg^{(i)} n \leq 1\}$$

Example 1.4.1.

$$\begin{aligned} \lg^* 2 &= 1 \\ \lg^* 4 &= 2 \\ \lg^* 16 &= 3 \\ \lg^* 65536 &= 4 \\ \lg^* 2^{65536} &= 5 \end{aligned}$$

Chapter 2

Recurrences

2.1 Master Method

The Master Method is used for solving recurrences of the form:

$$T(n) = aT(n/b) + f(n)$$

where $a \geq 1$ and $b > 1$ are constants, and $f(n)$ is an asymptotically positive function.

2.1.1 Solution Cases

- (1) If $f(n) = \mathcal{O}(n^{\log_b a - \epsilon})$ for some constant $\epsilon > 0$, then:

$$T(n) = \Theta(n^{\log_b a})$$

- (2) If $f(n) = \Theta(n^{\log_b a})$, then:

$$T(n) = \Theta(f(n) \lg n)$$

- (3) If $f(n) = \Omega(n^{\log_b a + \epsilon})$ for some constant $\epsilon > 0$, and if $af(n/b) \leq cf(n)$ for $c < 1$ and all sufficiently large n , then:

$$T(n) = \Theta(f(n))$$

Note. Here, $f(n)$ must be polynomially smaller or greater than $n^{\log_b a}$ in cases 1 and 3.

2.2 Changing Variables

We can use the technique of changing variables to simplify recurrences.

Example 2.2.1. Given $T(n) = 2T(\sqrt{n}) + \lg n$, let $n = 2^m$:

$$T(2^m) = 2T(2^{m/2}) + m$$

Put $T(2^m) = S(m)$:

$$S(m) = 2S(m/2) + m$$

Using the Master Method:

$$S(m) = \mathcal{O}(m \lg m)$$

Putting $m = \lg n$:

$$T(n) = \mathcal{O}(\lg n \cdot \lg \lg n)$$

Chapter 3

Sorting Algorithms

3.1 Insertion Sort

3.1.1 Algorithm Description

Insertion sort builds the final sorted array one item at a time. It is much less efficient on large lists than more advanced algorithms such as quicksort, heapsort, or merge sort.

Time Complexity:

- Best Case: $O(n)$
- Worst Case: $O(n^2)$
- Average Case: $O(n^2)$
- Maximum number of swaps required: $O(n^2)$

3.2 Merge Sort

3.2.1 Algorithm Description

Merge sort is a divide-and-conquer algorithm that divides the input array into two halves, calls itself for the two halves, and then merges the two sorted halves.

Recurrence Relation:

$$T(n) = 2T(n/2) + \Theta(n)$$

Time Complexity:

- Worst Case: $\Theta(n \lg n)$

3.3 Bubble Sort

3.3.1 Algorithm Description

Bubble sort repeatedly steps through the list, compares adjacent elements, and swaps them if they are in the wrong order.

Time Complexity:

- Average and Worst Case: $O(n^2)$

3.4 Selection Sort

3.4.1 Algorithm Description

Selection sort divides the input list into two parts: a sorted portion at the left end and an unsorted portion at the right end. The algorithm proceeds by finding the smallest element in the unsorted subarray and swapping it with the leftmost unsorted element.

Time Complexity:

- Worst Case: $O(n^2)$
- Average Case: $O(n^2)$
- Maximum swaps required: $O(n)$

3.5 Heap Sort

3.5.1 Max-Heapify

Max-Heapify is used for maintaining the heap property. The input is an array A and an index i into the array. When MAX-HEAPIFY is called, it is assumed that the binary trees rooted at $\text{LEFT}(i)$ and $\text{RIGHT}(i)$ are max-heaps, but that $A[i]$ may be smaller than its children.

Running Time: $O(\lg n)$ for a node of height h . For a node at $\text{RIGHT } R$, running time is $O(R)$.

3.5.2 Build-Max-Heap

Build-Max-Heap is used for building a max-heap from an array. It applies the MAX-HEAPIFY operation on all internal nodes from the leaf to the root node.

Running Time: $O(n)$

3.5.3 Heapsort

Heapsort uses the heap to sort the elements of the resultant array in non-decreasing order.

Running Time: $O(n \lg n)$

3.5.4 Heap-Extracts-Max

This operation removes the maximum element of the heap and returns it.

Running Time: $O(\lg n)$

3.5.5 Heap-Increase-Key

This operation increases the value at location i by the value key and then traverses the heap upward to maintain the heap property.

Running Time: $O(\lg n)$

3.5.6 Max-Heap-Insert

This operation inserts the element at the end of the heap and then traverses upward to maintain the heap property.

Running Time: $O(\lg n)$

3.6 Quick Sort

3.6.1 Algorithm Description

Quick sort uses a divide-and-conquer approach. It uses the PARTITION algorithm to partition the array into two sub-arrays. The PARTITION algorithm selects the last element of the input array as a pivot element and partitions the array such that the elements to the left are smaller than or equal to the pivot element, and the elements to the right of the pivot element are greater than it.

3.6.2 Performance Analysis

The running time of quicksort depends on whether the partitioning is balanced or unbalanced.

Worst Case Partitioning:

The worst case occurs when the partitioning routine produces one sub-problem with $n - 1$ elements and one with 0 elements.

$$T(n) = T(n - 1) + T(0) + \Theta(n) = T(n - 1) + \Theta(n) = \mathcal{O}(n^2)$$

Best Case Partitioning:

If the average-case running time of quicksort is much closer to the best case than to the worst case:

$$T(n) \leq 2T(n/2) + \Theta(n)$$

$$T(n) = \mathcal{O}(n \lg n)$$

Balanced Partitioning:

If the average-case running time of quicksort is much closer to the best case than to the worst case:

$$T(n) \leq T(9n/10) + T(n/10) + cn$$

Running Time: $\mathcal{O}(n \lg n)$

3.7 Counting Sort

3.7.1 Algorithm Description

Counting sort assumes that each of n input elements is an integer in the range 0 to k , for some integer k , where $k = O(n)$.

Running Time: $\Theta(n + k) = O(n)$

It is a stable sorting algorithm. This property is generally used for sorting suitable data.

3.8 Radix Sort

3.8.1 Algorithm Description

Radix sort uses a stable sorting algorithm.

Running Time: $\Theta(d(n + k))$, where d is the number of digits and k is 10 (0, 1, ..., 9) if d is constant and $k = O(n)$.

Running Time: $O(n) \rightarrow$ if d is constant.

3.9 Bucket Sort

3.9.1 Algorithm Description

Bucket sort divides the interval $[0, 1)$ into n equal-sized sub-intervals or buckets and then distributes the n input numbers into the buckets. Here, if n numbers are in the range $[0, 1)$, then:

Running Time: $\Theta(n)$

Chapter 4

Graph Algorithms

4.1 Representation of Graphs

A graph $G = (V, E)$ consists of a set of vertices V and a set of edges E .

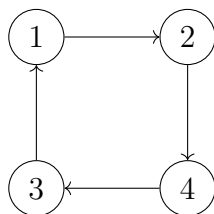
4.1.1 Adjacency List Representation

Adjacency list representation is used to represent sparse graphs, i.e., those for which $|E| \ll |V|^2$.

For an adjacency list representation, if G is a directed graph, the sum of all the adjacency lists is $|E|$. If G is an undirected graph, the sum is $2|E|$.

Amount of memory required for Adjacency List Representation: $\Theta(V + E)$

If the transpose of a directed graph $G = (V, E^T)$ is the graph $G^T = (V, E^T)$:



4.1.2 Adjacency Matrix Representation

Adjacency matrix representation is used for representing dense graphs, i.e., $|E| \leq |V|^2$.

For adjacency list representation, if G is a directed graph, the sum of all adjacency lists is $|E|$ and if G is an undirected graph, the sum is $2|E|$.

Amount of memory required: $\Theta(V + E)$

4.2 Breadth-First Search

Breadth-First Search (BFS) is used for searching a graph. It finds a minimum-spanning tree algorithm and Dijkstra's single-source shortest-path algorithm. We use ideas similar to those in BFS.

4.2.1 Algorithm Description

BFS produces a breadth-first tree with a root that contains all reachable nodes.

It works on both directed and undirected graphs.

To keep track of progress, breadth-first search colors each vertex white, gray, or black. All vertices start out white and may later become gray and then black.

- It assumes that the input graph $G = (V, E)$ is represented using adjacency lists.
- The color of each vertex u is stored in the variable $\text{color}[u]$, and the predecessor of u is stored in variable $\pi[u]$. The distance from the source s to vertex u is stored in $d[u]$. The algorithm also uses a FIFO queue Q to manage the set of gray vertices.

Running Time of BFS: $O(V + E)$

4.3 Depth-First Search

Depth-First Search (DFS) produces a depth-first forest composed of several depth-first trees.

Each vertex is initially white, is grayed when it is discovered in the search, and is blackened when it is finished.

It timestamps each vertex. Each vertex v has two timestamps: the first timestamp $d[v]$ records when v is first discovered (and grayed), and the second timestamp $f[v]$ records when the search finishes examining v 's adjacency list (and blackens v).

Vertex u is white before $d[u]$, gray between $d[u]$ and $f[u]$, and black thereafter.

4.3.1 Contains Two Methods

DFS-VISIT and DFS-G.

Running Time of DFS: $\Theta(V + E)$ (only for DAG)

4.4 Topological Sorting

Topological sorting is applicable only if the graph is acyclic. A topological sort of a graph is an ordering of its vertices along a horizontal line so that all directed edges go from left to right.

Running Time: $\Theta(V + E)$

Note. This is applicable only if the graph is a directed acyclic graph (DAG). We can sort the vertices in order of decreasing finishing times.

4.5 Strongly Connected Components

A graph is strongly connected if for every pair of vertices u, v : $u \rightarrow v$ and $v \rightarrow u$, i.e., u and v are reachable from each other.

STRONGLY-CONNECTED-COMPONENTS(G):

- (1) Call $\text{DFS}(G)$ to compute finishing times $f[u]$ for each vertex u .
- (2) Compute G^T .
- (3) Call $\text{DFS}(G^T)$, but in the main loop of DFS, consider the vertices in order of decreasing $f[u]$.
- (4) Output the vertices of each tree in the depth-first forest formed in Line 3 as a strongly-connected component.

Running Time: $\Theta(V + E)$

4.6 Minimum Spanning Trees

Kruskal's and Prim's Algorithms are used to find minimum spanning trees from a weighted graph. Both are greedy algorithms.

4.6.1 Kruskal's Algorithm

Kruskal's algorithm is a greedy algorithm because at each step it adds to the forest an edge of least possible weight.

Running Time of Kruskal's algorithm for a graph $G = (V, E)$ depends on the implementation of the disjoint-set data structure.

Running Time: $O(E \lg E)$. Since $|E| < |V|^2 \Rightarrow \lg |E| = O(\lg V)$.

Running Time: $O(E \lg V)$

If we use a binary min-heap, then:

Running Time: $O(V \lg V + E \lg V) = O(E \lg V)$

If we use Fibonacci heap, then:

Running Time: $O(E + V \lg V)$

4.6.2 Prim's Algorithm

Prim's algorithm operates much like Dijkstra's algorithm for finding shortest paths in a graph.

The edges in the set A always form a single tree. The tree starts from an arbitrary root vertex and grows until the tree spans all the vertices in V . At each step, a light edge is added to the tree A that connects A to an isolated vertex of $G_A = (V, A)$.

Running Time:

- (i) Using Binary Min-Heap:

$$O((V + E) \lg V) = O(E \lg V)$$

(ii) Using Fibonacci Min-Heap:

$$O(E + V \lg V)$$

4.7 Single Source Shortest Paths

4.7.1 Bellman-Ford Algorithm

The Bellman-Ford algorithm solves the single-source shortest-path problem in which edge weights may be negative.

It uses relaxation to find shortest paths. It returns TRUE if the graph contains no negative-weight cycles that are reachable from the source.

Running Time: $O(VE)$

Note: This algorithm does not find any negative-weight cycle. It only finds a relaxable *-ve* weight cycle. If such a cycle is reachable from the source, it returns FALSE, indicating that no solution exists for the source.

Single Source Shortest Paths in Directed Acyclic Graphs:

By relaxing the edges of a weighted DAG according to a topological sort of its vertices, we can compute shortest paths from a single source in $\Theta(V + E)$ time.

4.7.2 Dijkstra's Algorithm

Dijkstra's algorithm solves the single-source shortest-paths problem in which all edge weights are non-negative.

It maintains a set S of vertices whose shortest-path weights from the source have been determined. The algorithm repeatedly selects the vertex $u \in V - S$ with the minimum shortest-path estimate, adds u to S , and relaxes all edges leaving u .

The running time of Dijkstra's algorithm depends on how the min-priority queue is implemented.

Running Time:

(i) Using Binary Min-Heap:

$$O((V + E) \lg V) = O(E \lg V)$$

(ii) Using Fibonacci Min-Heap:

$$O(E + V \lg V)$$

Chapter 5

Data Structures

5.1 Arrays

A linear array is a list of a finite number n of homogeneous data elements such that:

- (a) The elements of the array are referenced respectively by an index set consisting of n consecutive numbers.
- (b) The elements of the array are stored respectively in successive memory locations.

Length of the array:

$$\text{Length} = UB - LB + 1$$

5.1.1 Representation of Linear Array in Memory

$$\text{LOC}(A[K]) = \text{Base}(A) + w(K - LB)$$

where w is the size of word/memory cell, K is the index of the element in the array, and LOC is the location of an element.

5.1.2 Representation of 2-D Array in Memory

(a) **Row-Major Order:**

$$\text{LOC}(A[J, K]) = \text{Base}(A) + w[N(J - 1) + (K - 1)]$$

(b) **Column-Major Order:**

$$\text{LOC}(A[J, K]) = \text{Base}(A) + w[M(K - 1) + (J - 1)]$$

where A is an $M \times N$ array and $\text{Base}(A)$ is the base address of A .

5.2 Stacks

In a stack, the element deleted from the set is the one most recently inserted. The stack implements a Last-In, First-Out (LIFO) policy.

The INSERT operation on a stack is often called PUSH, and the DELETE operation, which does not take an element argument, is often called POP.

We can implement a stack of at most n elements with an array $S[1..n]$. The array has an attribute $\text{top}[S]$ that indexes the most recently inserted element. When $\text{top}[S] = 0$, the stack contains no elements and is empty.

5.2.1 Arithmetic Expressions

- (a) **Infix Notation:** $A + B$, $C - D$, $E * F$, G/H
- (b) **Prefix Notation** or **Polish Notation:** $+AB$, $-CD$, $*EF$, $/GH$
- (c) **Reverse Polish Notation** or **Postfix Notation:** $AB+$, $CD-$, $EF*$, $GH/$

5.3 Queues

In a queue, the element deleted is always the one that has been in the set for the longest time. The queue implements a First-In, First-Out (FIFO) policy.

The insert operation of a queue is called ENQUEUE and the delete operation is called DEQUEUE.

The queue has an attribute $\text{Read}[Q]$ that indexes, or points to, its head. The attribute $\text{tail}[Q]$ indexes the next location at which a newly existing element will be inserted into the queue.

When $\text{Read}[Q] = \text{tail}[Q]$, the queue is empty.

Initially, we have $\text{Read}[Q] = \text{tail}[Q] = 1$.

When $\text{Read}[Q] = \text{tail}[Q] + 1$, the queue is full.

5.4 Linked Lists

A linked list is a linear collection of data elements, called nodes, where the linear order is given by means of pointers.

A doubly linked list L is an object with a key field and two other pointer fields: next and prev. Given an element x in the list, $\text{next}[x]$ points to its successor in the linked list, and $\text{prev}[x]$ points to its predecessor. If $\text{prev}[x] = \text{NIL}$, the element x has no predecessor and is therefore the first element, or head, of the list. If $\text{next}[x] = \text{NIL}$, the element x has no successor and is therefore the last element, or tail, of the list. An attribute $\text{head}[L]$ points to the first element of the list. If $\text{head}[L] = \text{NIL}$, the list is empty.

- If a list is singly linked, we omit the prev pointer in each element.
- If a list is sorted, the linear order of the list corresponds to the linear order of keys stored in elements of the list. The minimum element is then the head of the list.
- In a circular list, the prev pointer of the head of the list points to the tail, and the next pointer of the tail of the list points to the head.

5.5 Binary Trees

5.5.1 Definition

Binary trees T and T' are said to be similar if they have the same structure or, in other words, if they have the same shape. The trees are said to be copies if they are similar and if they have the same contents at corresponding nodes.

A node L is called a descendant of a node N (and N is called an ancestor of L) if there is a succession of children from N to L . In particular, L is called a left or right descendant of N according to whether L belongs to the left or right subtree of N .

5.5.2 Complete Binary Trees

The tree T is said to be complete if all its levels, except possibly the last, have the maximum number of possible nodes and if all the nodes at the last level appear as far left as possible.

5.5.3 Extended Binary Tree (2-Tree)

A binary tree T is said to be a 2-tree or an extended binary tree if each node N has either 0 or 2 children. In such a case, the nodes with 2 children are called internal nodes, and the nodes with 0 children are called external nodes.

In an extended k -ary tree, the number of leaves with n internal nodes is:

$$L = n(k - 1) + 1$$

5.6 Binary Search Trees

A Binary Search Tree (BST) is organized in a binary tree. In addition to a key field, each node contains fields left, right, and p that point to the nodes corresponding to its left child, right child, and its parent.

The keys in BST always satisfy the Binary-Search-Tree Property: Let x be a node in BST. If y is a node in the left subtree of x , then $\text{key}[y] \leq \text{key}[x]$. If y is a node in the right subtree of x , then $\text{key}[x] \leq \text{key}[y]$.

The in-order walk of BST prints the keys in non-decreasing order.

It takes $O(h)$ time to walk an n -node BST, since after the initial call, the procedure is called recursively exactly twice for each node in the tree—once for its left child and once for its right child.

5.6.1 Operations on a BST

- (1) **Searching:** Running Time = $O(h)$, where h is the height of the tree.
- (2) **Minimum and Maximum:** To find the minimum, follow left child pointers and to find the maximum, follow right child pointers from the root until a NIL is encountered.

Running Time = $O(h)$.

Number of different binary search trees: Let b_n denote the number of different binary trees with n nodes, then:

$$b_n = \frac{n-1}{\sum_{k=0}^{n-1}} b_k \cdot b_{n-1-k} \quad \text{where } b_0 = 1 \text{ \& } b_1 = 1$$

$$\Rightarrow n \geq 2^h$$

5.7 AVL Search Trees

An AVL tree is a binary search tree that is height-balanced. For each node x , the heights of the left and right subtrees of x differ by at most 1.

$$B.F. = h(T^L) - h(T^R)$$

where $B.F.$ is the Balance Factor and T^L and T^R are the heights of the left and right subtrees.

For an AVL tree:

$$|h(T^L) - h(T^R)| \leq 1$$

5.7.1 Representation

AVL search trees like BST are represented using a linked representation. However, every node registers its balance factor.

5.7.2 Insertion in AVL Search Tree

Inserting an element into an AVL tree in its first phase is similar to that of the one used in a binary search tree. However, if after insertion of the element, the balance factor of any node in the tree is affected so as to render the binary search tree unbalanced, we resort to techniques called Rotations to restore the balance of the tree.

To perform rotations, it is necessary to identify a specific node A whose $BF(A)$ is neither 0, 1, or -1, which is the nearest ancestor to the inserted node on the path from the inserted node to the root.

5.7.3 Types of Rotations

1. **LL Rotation:** Inserted node is in the left subtree of the left subtree of node A .
2. **RR Rotation:** Inserted node is in the right subtree of the right subtree of node B .
3. **LR Rotation:** Inserted node is in the right subtree of the left subtree of node A .
4. **RL Rotation:** Inserted node is in the left subtree of the right subtree of node A .

5.7.4 LR and RL Rotations

The balancing methodology of LR and RL rotations are similar in nature but are mirror images of one another.

LR can be accomplished by RR followed by LL rotation, and RL can be accomplished by LL followed by RR rotation.

5.8 B-Trees

A B-tree T is a rooted tree having the following properties:

- (1) Every node x has the following fields:
 - (a) $n[x]$, the number of keys currently stored in node x .
 - (b) The $n[x]$ keys themselves, stored in non-decreasing order, so that $\text{key}_1[x] \leq \text{key}_2[x] \leq \dots \leq \text{key}_{n[x]}[x]$.
 - (c) $\text{leaf}[x]$, a boolean value that is true if x is a leaf and false if x is an internal node.
- (2) Each internal node x also contains $n[x] + 1$ pointers $c_1[x], c_2[x], \dots, c_{n[x]+1}[x]$ to its children. Leaf nodes have no children, so their c_i fields are undefined.
- (3) The keys $\text{key}_j[x]$ separate the ranges of keys stored in each subtree: if k_i is any key stored in the subtree with root $c_i[x]$, then:

$$k_1 \leq \text{key}_1[x] \leq k_2 \leq \text{key}_2[x] \leq \dots \leq \text{key}_{n[x]}[x] \leq k_{n[x]+1}$$

- (4) All leaves have the same depth, which is the tree's height h .
- (5) There are lower and upper bounds on the number of keys a node can contain. It is represented by an integer $t \geq 2$ called the minimum degree of the B-tree.

5.8.1 Minimum degree of the B-tree

- (a) Every node other than the root must have at least $t - 1$ keys. Every internal node other than the root thus has at least t children. If the tree is non-empty, the root must have at least one key.
- (b) Every node can contain at most $2t - 1$ keys. Therefore, an internal node can have at most $2t$ children. A node is full if it contains $(2t - 1)$ keys.

5.8.2 Height of a B-tree

If $n \geq 1$, then for any n -key B-tree T of height h and minimum degree $t \geq 2$:

$$h \leq \log_t \left(\frac{n+1}{2} \right)$$

or

$$n \geq 2t^h - 1$$

5.8.3 Searching a B-Tree

The running time of a B-tree search is $O(t \log_t n)$ and the number of keys in a page is less than $2t$.

$$\text{So, total search time} = O(t \log_t n) \approx O(\log_t n)$$

and the number of keys in a page less than $2t$ is:

$$= O(t)$$

5.8.4 Inserting a Key into a B-Tree

To insert a key into a B-tree, we travel down the tree searching for the position where the new key belongs, we split each full node we come to along the way (including the leaf itself). Thus, whenever we want to split a full node y , we are assured that its parent is not full.

Running Time: $O(t \log_t n)$

5.8.5 Deleting a Key from a B-Tree

Here, we must ensure that a node doesn't get too small during deletion (except the root).

Procedure:

- (1) If the key k is in node x and x is a leaf, delete the key k from x .
- (2) If the key k is in node x and x is an internal node, do the following:
 - (a) If the child y that precedes k in node x has at least t keys, then find the predecessor k' of k in the subtree rooted at y . Recursively delete k' and replace k by k' in x .
 - (b) Symmetrically, if the child z that follows k in node x has at least t keys, then find the successor k' of k in the subtree rooted at z . Recursively delete k' and replace k by k' in x .
 - (c) Otherwise, if both y and z have only $t - 1$ keys, merge k and all of z into y , so that x loses both k and the pointer to z , and y now contains $2t - 1$ keys. Then free z and recursively delete k from y .
- (3) If the key k is not present in internal node x , determine the root $c_i[x]$ of the appropriate subtree that must contain k . If $c_i[x]$ has only $t - 1$ keys, execute step 3a or 3b as necessary to guarantee that we descend to a node containing at least t keys. Then finish by recursing on the appropriate child of x .
 - (a) If $c_i[x]$ has only $t - 1$ keys but has an immediate sibling with at least t keys, give $c_i[x]$ an extra key by moving a key from x down into $c_i[x]$, moving a key from $c_i[x]$'s immediate left or right up into x , and moving the appropriate child pointer from the sibling into $c_i[x]$.
 - (b) If $c_i[x]$ and both of $c_i[x]$'s immediate siblings have $t - 1$ keys, merge $c_i[x]$ with one sibling, which involves moving a key from x down into the new merged node to become the median key for that node.

5.9 Hash Tables

5.9.1 Direct Address Tables

Direct address tables work well when the universe U of keys is reasonably small.

Let universe $U = \{0, 1, \dots, m-1\}$, where m is not too large.

To represent the dynamic set, we use an array, or Direct-address table, denoted by $T[0..m-1]$ in which each position, or slot, corresponds to a key in the universe U .

5.9.2 Hash Tables

With direct addressing, an element with key k is stored in slot k . With hashing, this element is stored in slot $h(k)$, where h is the hash function. The hash function h maps the universe U of keys into the slots of a hash table $T[0..m-1]$:

$$h : U \rightarrow \{0, 1, \dots, m-1\}$$

Here:

$$k \rightarrow U \rightarrow \{0, 1, \dots, m-1\}$$

5.9.3 Hash Functions

A good hash function satisfies the assumption of simple uniform hashing: each key is equally likely to hash to any of the m slots independently of where any other key has hashed to.

A hash function h must be deterministic in that a given input k should always produce the same output $h(k)$.

5.9.4 Types of Hashing Schemes

Division Method

$$h(k) = k \mod m$$

A prime not too close to an exact power of 2 is often a good choice for m .

Multiplication Method

$$h(k) = \lfloor m(kA \mod 1) \rfloor$$

where $0 < A < 1$ and $kA \mod 1$ is the fractional part.

5.9.5 Collision Resolution

When two keys hash to the same slot, this situation is called collision. Collision is also called an open hashing.

Collision Resolution by Chaining

In chaining, we put all the elements that hash to the same slot in a linked list as shown in the figure.

Running Time: $O(1 + \log_2 m)$

Open Addressing

In open addressing, all elements are stored in the hash table itself.

To perform insertion using open addressing, we successively examine or probe the hash table until we find an empty slot to put the key.

The algo for searching for key k probes the same sequence of slots used during insertion.

Deletion from an open-address hash table is difficult. When we delete a key from slot i , we can't simply mark that slot as empty by storing NIL in it. Doing so might make it impossible to retrieve any key k during whose insertion we had probed slot i and found it occupied.

We use a small secondary table hash table S_j with an associated hash function h_j .

5.9.6 Types of Probing

Linear Probing

Given an ordinary hash function $h' : U \rightarrow \{0, 1, \dots, m-1\}$, which we refer to as an auxiliary hash function, the method of linear probing uses the hash function:

$$h(k, i) = (h'(k) + i) \mod m$$

for $i = 0, 1, \dots, m-1$.

Quadratic Probing

Quadratic probing uses a hash function of the form:

$$h(k, i) = (h'(k) + c_1i + c_2i^2) \mod m$$

where h' is an auxiliary function, c_1 and $c_2 \neq 0$ are auxiliary constants, and $i = 0, 1, \dots, m-1$.

Double Hashing

Double hashing uses the hash function of the form:

$$h(k, i) = (h_1(k) + ih_2(k)) \mod m$$

where h_1 and h_2 are auxiliary functions.

5.9.7 Perfect Hashing

Perfect hashing is a technique that allows a search in $O(1)$ worst case.

Instead of making a list of the keys hashing to slot j , however, we use a small secondary hash table S_j with an associated hash function h_j .

If we use a binary min-heap, then:

Running Time: $O(V \lg V + E \lg V) = O(E \lg V)$

If we use Fibonacci heap, then:

Running Time: $O(E + V \lg V)$

Chapter 6

Additional Topics

6.1 Tower of Hanoi Problem

Suppose there are three pegs labeled A, B, and C and suppose on peg A there is a finite number n of disks with decreasing size. The object of the game is to move the disks from peg A to peg C using peg B as an auxiliary peg.

- (a) Only one disk may be moved at a time, i.e., the top disk on any peg.
- (b) At no time a larger disk be placed on a smaller disk.

6.1.1 Algorithm

TOWER(N , BEG, AUX, END):

- (1) If $N = 1$, then:
 - (a) Write BEG $\rightarrow$ END
 - (b) Return
- (2) Call TOWER($N - 1$, BEG, END, AUX)
- (3) Write BEG $\rightarrow$ END
- (4) Call TOWER($N - 1$, AUX, BEG, END)
- (5) Return

Complexity:

$$T(n) = 2T(n - 1) + \Theta(1) = \Theta(2^n)$$

6.2 Assembly Line Scheduling

Assembly line scheduling is a problem in dynamic programming.

6.3 Longest Common Subsequence

The Longest Common Subsequence (LCS) problem is a classic problem in dynamic programming.

6.4 Matrix Chain Multiplication

Matrix chain multiplication is an optimization problem that determines the most efficient way to multiply a given sequence of matrices.

6.4.1 Solution

Given matrices:

$$M_1 = (10 \times 100)$$

$$M_2 = (100 \times 20)$$

$$M_3 = (20 \times 5)$$

$$M_4 = (5 \times 80)$$

The number of multiplications required = 19,000

6.4.2 Inversions

Let $A[1 \dots n]$ be an array of n distinct numbers. If $i < j$ and $A[i] > A[j]$, then the pair (i, j) is called an inversion of A .

The maximum number of inversions in an array with n distinct elements is $\frac{n(n-1)}{2} = C_2 = \binom{n}{2}$.

As the number of inversions increases, the running time of insertion sort also increases.

To determine the number of inversions in any permutation on n -elements takes $O(n \lg n)$ time in the worst case (using modified merge sort).